

LAPORAN MODUL 3 PARADIGMA PEMGROGRAMAN



Disusun oleh :

Nama : Irvan Cahya Nugraha

NIM : 245410034

Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 3

METHOD

A. TUJUAN PRAKTIKUM

Dapat membuat dan menggunakan berbagai method

B. DASAR TEORI

Method

Sebuah method menjelaskan behaviour dari sebuah object. Method juga dikenal sebagai fungsi atau prosedur. Pendeklarasian method biasa dituliskan seperti berikut ini :

```
<modifier> <returnType> <name>(<parameter>*) {  
<statement>*  
}
```

dimana,

<modifier> dapat menggunakan beberapa *modifier* yang berbeda

<returnType> dapat berupa seluruh tipe data, termasuk *void*

<name> *identifier* atas *class*

<parameter> ::= <tipe_parameter> <nama_parameter>[,]

Method Tanpa Return Value

Jenis method ini ditandai dengan *return type* yang berupa *void* dan pada bagian *statement* tidak terdapat keyword **return**. Pada pemrograman berorientasi obyek method jenis ini digunakan untuk membuat method *mutator*. Nama untuk method *mutator* sebaiknya diawali dengan kata set, misalnya: setNama(), setAlamat().

Method Dengan Return Value

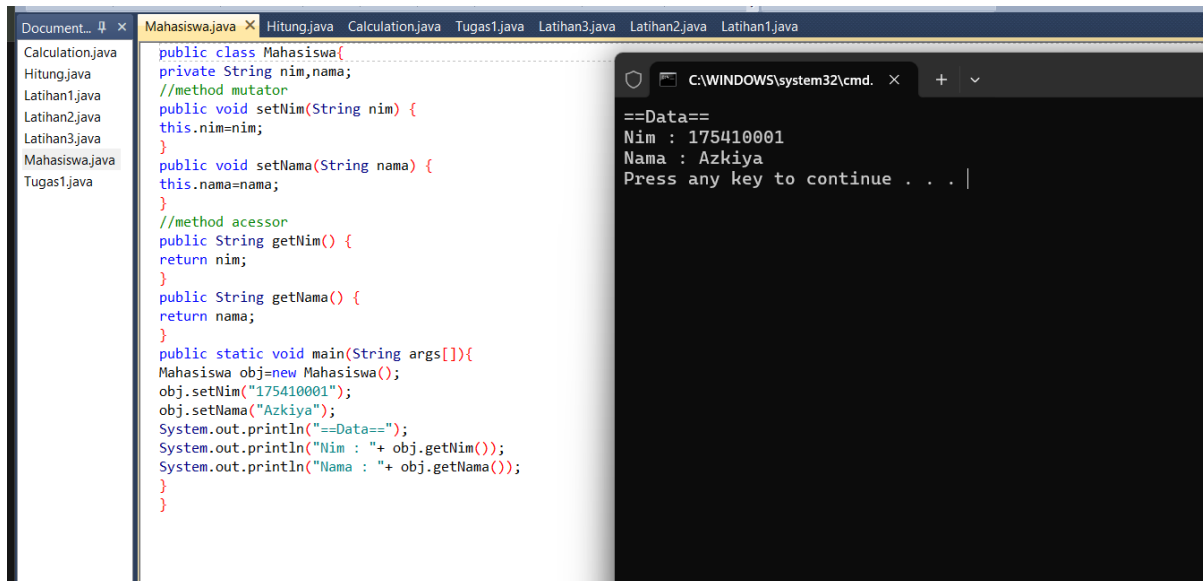
Jenis method ini ditandai dengan *return type* selain *void* dan pada bagian *statement* terdapat keyword **return**. Pada pemrograman berorientasi obyek method jenis ini digunakan untuk membuat method *acessor*, dimana method *acessor* fungsinya adalah untuk membaca/mendapatkan nilai suatu atribut. Nama untuk method *acessor* sebaiknya diawali dengan kata get, misalnya: getNama(), getAlamat().

Method Overloading

Bahasa java mendukung method *overloading* , java dapat membedakan beberapa *method* dengan nama yang sama di dalam sebuah kelas namun parameternya berbeda. Hal ini sangat menguntungkan karena memudahkan kita dalam mengingat nama method, bayangkan bila program pada class Gambar harus diberi nama drawInteger(int i), drawString(String s), drawDouble(double d). Method *overloading* dibedakan oleh jumlah dan jenis tipe data parameternya

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1



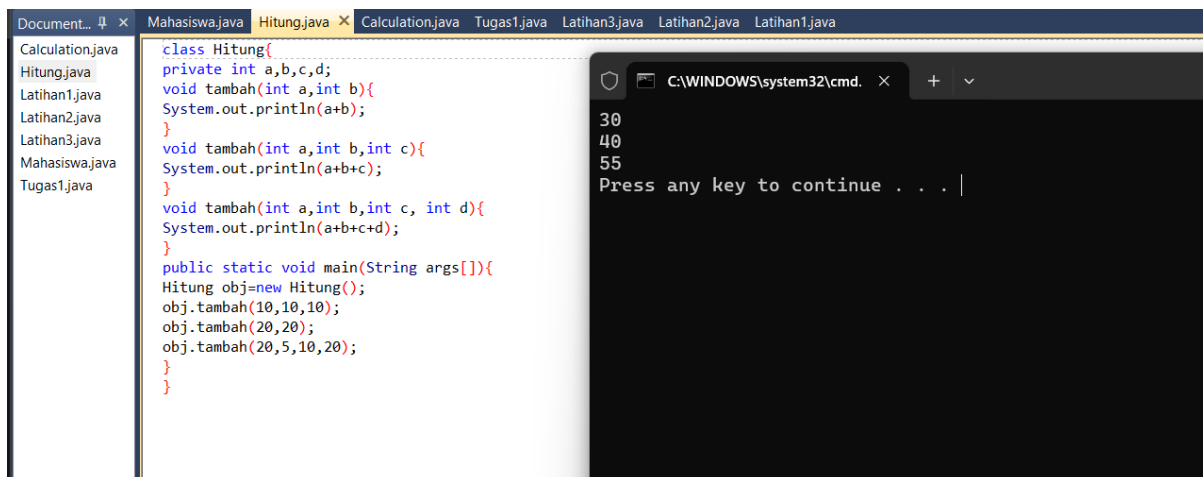
```
Document... x Mahasiswa.java Hitung.java Calculation.java Tugas1.java Latihan3.java Latihan2.java Latihan1.java
Calculation.java
Hitung.java
Latihan1.java
Latihan2.java
Latihan3.java
Mahasiswa.java
Tugas1.java

public class Mahasiswa{
    private String nim,nama;
    //method mutator
    public void setNim(String nim) {
        this.nim=nim;
    }
    public void setName(String nama) {
        this.nama=nama;
    }
    //method accessor
    public String getNim() {
        return nim;
    }
    public String getName() {
        return nama;
    }
    public static void main(String args[]){
        Mahasiswa obj=new Mahasiswa();
        obj.setNim("175410001");
        obj.setName("Azkiya");
        System.out.println("==Data==");
        System.out.println("Nim : "+ obj.getNim());
        System.out.println("Nama : "+ obj.getName());
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
==Data==
Nim : 175410001
Nama : Azkiya
Press any key to continue . . . |
```

Pembahasan : Praktik 1 menggunakan method dimulai dengan membuat atribut private string nim nama, kemudian membuat setter baru pada `this.nim=nim;` dan juga nama, method ditambahkan method `getNim` dan `getName` yang mereturn value. Pada main dibuat objek baru `obj`, lalu `obj` dipanggil dengan method yaitu `obj.setNim("2454")` maka `setNim` akan menyimpan value tersebut dan untuk menampilkan dipakai `system.out.getNim`

Praktik 2



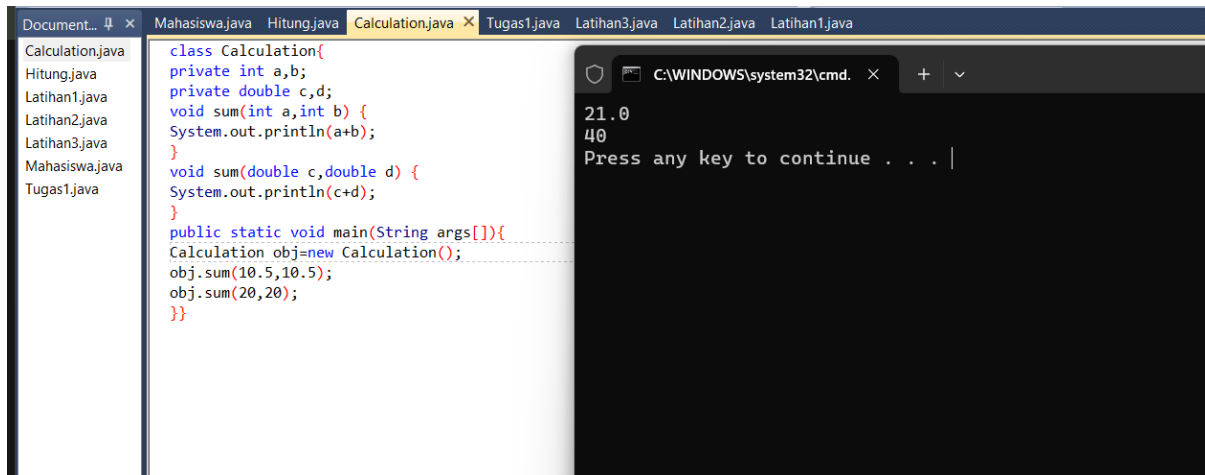
```
Document... x Mahasiswa.java Hitung.java Calculation.java Tugas1.java Latihan3.java Latihan2.java Latihan1.java
Calculation.java
Hitung.java
Latihan1.java
Latihan2.java
Latihan3.java
Mahasiswa.java
Tugas1.java

class Hitung{
    private int a,b,c,d;
    void tambah(int a,int b){
        System.out.println(a+b);
    }
    void tambah(int a,int b,int c){
        System.out.println(a+b+c);
    }
    void tambah(int a,int b,int c, int d){
        System.out.println(a+b+c+d);
    }
    public static void main(String args[]){
        Hitung obj=new Hitung();
        obj.tambah(10,10,10);
        obj.tambah(20,20);
        obj.tambah(20,5,10,20);
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
30
40
55
Press any key to continue . . . |
```

Pembahasan : Pada praktik 2 method overloading, yaitu membuat beberapa method dengan nama yang sama yaitu `tambah()`, tapi parameternya berbeda-beda. Di sini ada `tambah(int a, int b)`, lalu `tambah(int a, int b, int c)`, dan juga `tambah(int a, int b, int c, int d)`. Walaupun namanya sama, Java tetap bisa bedain karena jumlah parameternya beda. Cara kerjanya, ketika objek `Hitung obj = new Hitung();` dipanggil lalu `obj.tambah(20,20);`, maka yang jalan adalah method dengan 2 parameter. Kalau `obj.tambah(10,10,10);` maka yang dipakai method tiga parameter. Jadi praktik ini nunjukkan kalau satu nama method bisa punya fungsi berbeda tergantung inputnya, tanpa bikin nama method lain.

Praktik 3



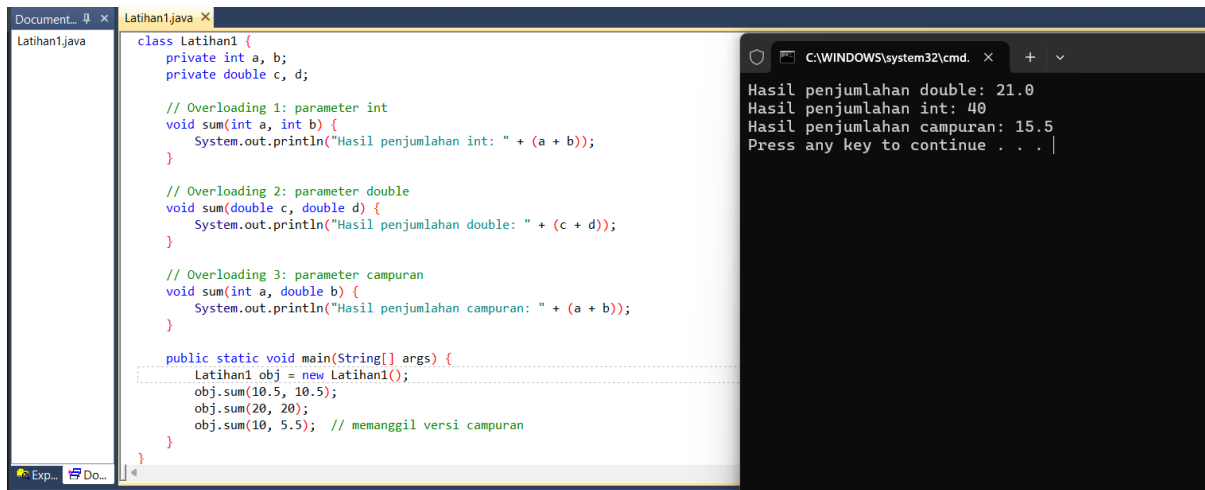
```
class Calculation{
    private int a,b;
    private double c,d;
    void sum(int a,int b) {
        System.out.println(a+b);
    }
    void sum(double c,double d) {
        System.out.println(c+d);
    }
    public static void main(String args[]){
        Calculation obj=new Calculation();
        obj.sum(10.5,10.5);
        obj.sum(20,20);
    }
}
```

```
21.0
40
Press any key to continue . . . |
```

Pembahasan : Pada praktik 3 overloading dilanjutkan tapi fokus pada perbedaan tipe parameter. Di kelas Calculation2, dibuat dua method sum(), tapi satu memakai int dan satunya pakai double. Jadi walaupun jumlah parameternya sama, Java tetap anggap beda karena tipe datanya berbeda. Saat dijalankan `obj.sum(10.5, 10.5);` maka method double yang dijalankan, tetapi `obj.sum(20,20);` memakai method int. Praktik ini memperjelas kalau overloading tidak selalu harus menambah parameter, bisa juga hanya beda tipe. Ini penting supaya program fleksibel menerima input berbeda tapi tetap satu nama method.

LATIHAN

Latihan 1



```
class Latihan1 {
    private int a, b;
    private double c, d;

    // Overloading 1: parameter int
    void sum(int a, int b) {
        System.out.println("Hasil penjumlahan int: " + (a + b));
    }

    // Overloading 2: parameter double
    void sum(double c, double d) {
        System.out.println("Hasil penjumlahan double: " + (c + d));
    }

    // Overloading 3: parameter campuran
    void sum(int a, double b) {
        System.out.println("Hasil penjumlahan campuran: " + (a + b));
    }

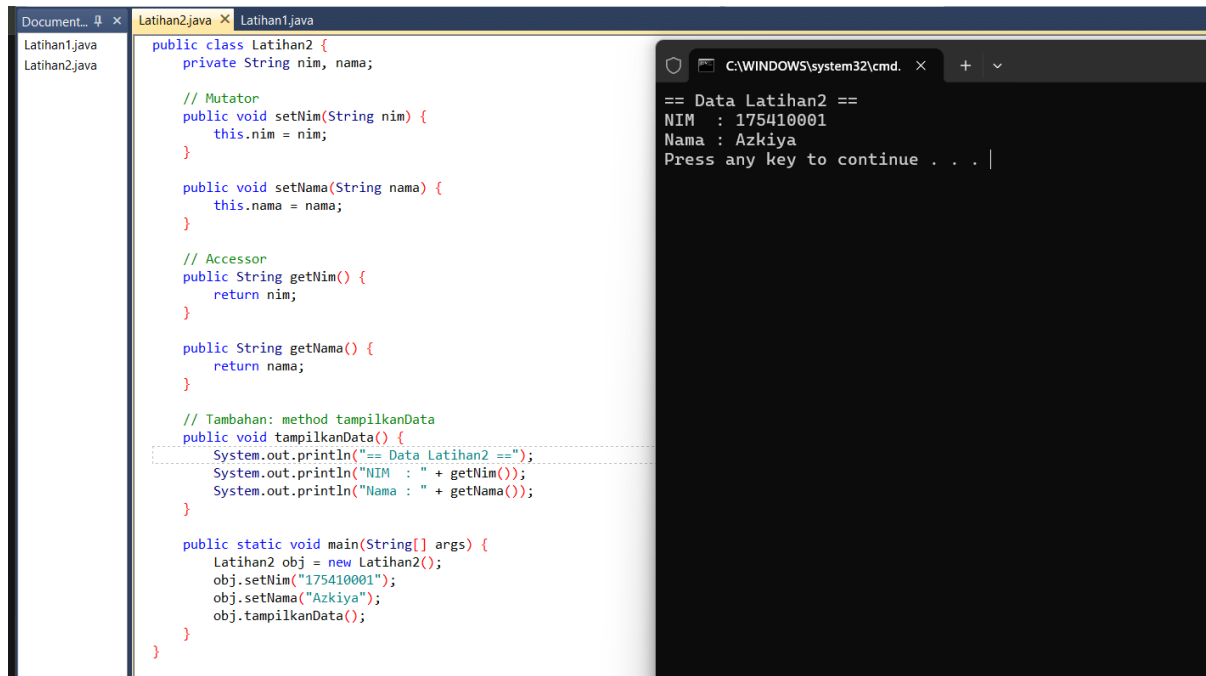
    public static void main(String[] args) {
        Latihan1 obj = new Latihan1();
        obj.sum(10.5, 10.5);
        obj.sum(20, 20);
        obj.sum(10, 5.5); // memanggil versi campuran
    }
}
```

```
Hasil penjumlahan double: 21.0
Hasil penjumlahan int: 40
Hasil penjumlahan campuran: 15.5
Press any key to continue . . . |
```

Pembahasan : Pada latihan 1 dilakukan modifikasi dari praktik 3 dengan menambahkan satu method sum lagi tapi kali ini pakai tipe campuran seperti `sum(int a, double b)`. Secara konsep ini masih tetap overloading karena nama method tetap sum, tapi parameter yang diterima berbeda, baik dari jumlah maupun jenis tipe datanya. Saat objek dibuat `Latihan1 obj = new Latihan1();`, pemanggilan `obj.sum(10.5, 10.5);` akan pakai method double-double, `obj.sum(20,20);` akan panggil method integer, dan `obj.sum(10,5.5);` otomatis memilih method campuran. Hal ini membuktikan bahwa Java bisa memilih method otomatis berdasarkan tipe

data yang dikirim, tanpa programmer harus tulis nama method berbeda. Ide ini bikin kode lebih rapi dan mudah digunakan kembali.

Latihan 2



```
Document... x Latihan2.java x Latihan1.java
Latihan1.java
Latihan2.java

public class Latihan2 {
    private String nim, nama;

    // Mutator
    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setName(String nama) {
        this.nama = nama;
    }

    // Accessor
    public String getNim() {
        return nim;
    }

    public String getName() {
        return nama;
    }

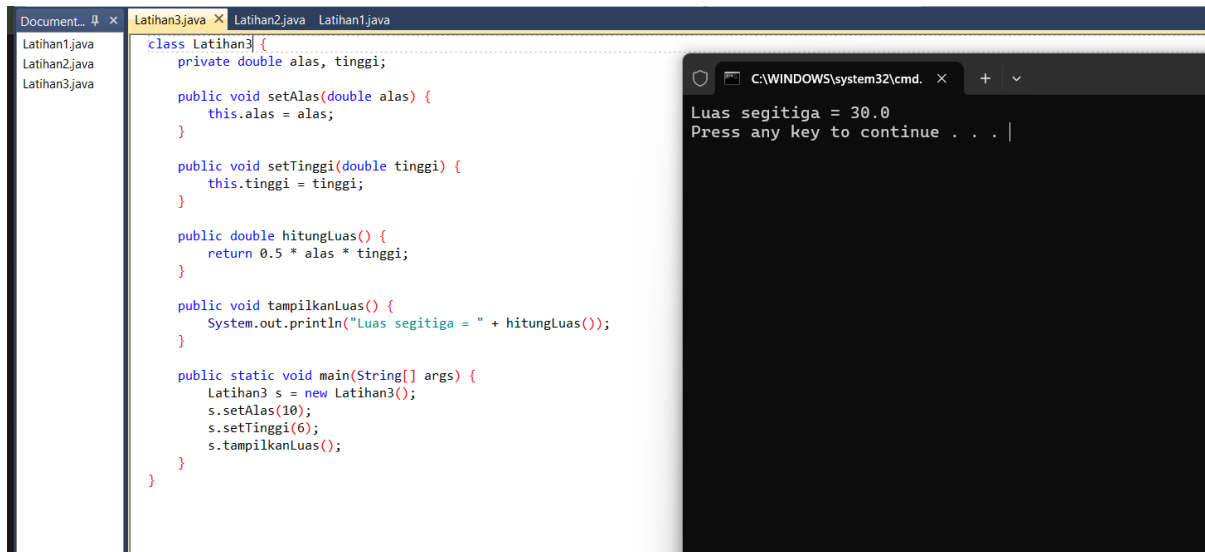
    // Tambahan: method tampilkanData
    public void tampilkanData() {
        System.out.println("== Data Latihan2 ==");
        System.out.println("NIM : " + getNim());
        System.out.println("Nama : " + getName());
    }

    public static void main(String[] args) {
        Latihan2 obj = new Latihan2();
        obj.setNim("175410001");
        obj.setName("Azkiya");
        obj.tampilkanData();
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
== Data Latihan2 ==
NIM : 175410001
Nama : Azkiya
Press any key to continue . . . |
```

Pembahasan : Pada latihan 2 dilakukan modifikasi pada kelas Mahasiswa dengan menambahkan satu method baru yaitu tampilkanData(). Kalau sebelumnya data hanya bisa diambil lewat getNim() atau getName(), sekarang method tampilkanData dibuat sebagai shortcut untuk menampilkan semua informasi langsung dalam satu pemanggilan. Di dalam method tampilkanData() dipanggil getNim() dan getName(), lalu dicetak dengan System.out.println. Tujuannya supaya program bisa lebih ringkas ketika menampilkan data objek. Jadi latihan ini memperjelas perbedaan antara accessor (get) untuk mengambil nilai dan method tampilan untuk output langsung ke layar. Ini juga menunjukkan pembungkusan (enkapsulasi) tetap dijaga karena data tidak diakses langsung dari atribut.

Latihan 3



```
class Latihan3 {
    private double alas, tinggi;

    public void setAlas(double alas) {
        this.alas = alas;
    }

    public void setTinggi(double tinggi) {
        this.tinggi = tinggi;
    }

    public double hitungLuas() {
        return 0.5 * alas * tinggi;
    }

    public void tampilkanLuas() {
        System.out.println("Luas segitiga = " + hitungLuas());
    }

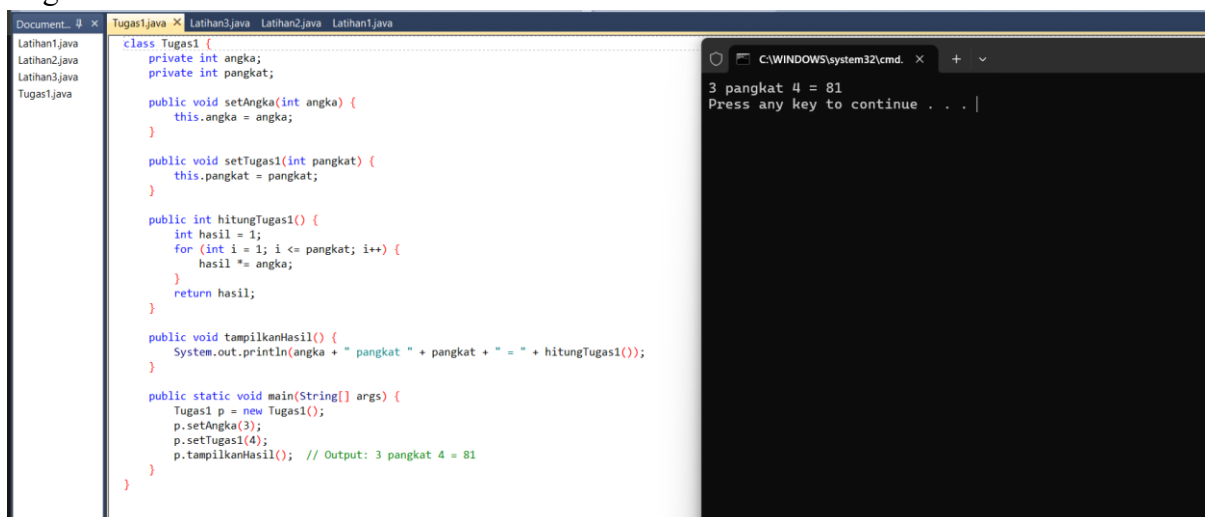
    public static void main(String[] args) {
        Latihan3 s = new Latihan3();
        s.setAlas(10);
        s.setTinggi(6);
        s.tampilkanLuas();
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
Luas segitiga = 30.0
Press any key to continue . . . |
```

Pembahasan : Pada Tugas 3 dibuat kelas Latihan3 untuk menghitung luas segitiga. Kelas ini memiliki dua atribut privat: alas dan tinggi, yang diisi lewat method `setAlas()` dan `setTinggi()`. Perhitungan luas dilakukan oleh method `hitungLuas()` dengan rumus $0.5 * \text{alas} * \text{tinggi}$, lalu hasilnya ditampilkan lewat `tampilkanLuas()`. Di method `main`, dibuat objek, nilai alas dan tinggi di-set, lalu luas dicetak. Program ini menunjukkan dasar OOP: enkapsulasi, method, dan penggunaan objek.

D. PEMBAHASAN TUGAS

Tugas 1



```
class Tugas1 {
    private int angka;
    private int pangkat;

    public void setAngka(int angka) {
        this.angka = angka;
    }

    public void setTugas1(int pangkat) {
        this.pangkat = pangkat;
    }

    public int hitungTugas1() {
        int hasil = 1;
        for (int i = 1; i <= pangkat; i++) {
            hasil *= angka;
        }
        return hasil;
    }

    public void tampilkanHasil() {
        System.out.println(angka + " pangkat " + pangkat + " = " + hitungTugas1());
    }

    public static void main(String[] args) {
        Tugas1 p = new Tugas1();
        p.setAngka(3);
        p.setTugas1(4);
        p.tampilkanHasil(); // Output: 3 pangkat 4 = 81
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
3 pangkat 4 = 81
Press any key to continue . . . |
```

Pembahasan : Pada tugas 1 dibuat sebuah program untuk menghitung nilai pangkat dari suatu bilangan menggunakan atribut dan operasi dalam class. Atribut yang dipakai adalah angka dan pangkat, kemudian operasi perhitungannya dibuat dalam method `hitungTugas1()` menggunakan perulangan `for`. Setiap iterasi akan mengalikan nilai angka ke dalam variabel `hasil` secara bertahap sampai sesuai dengan nilai pangkat. Setelah itu method `tampilkanHasil()` digunakan untuk menampilkan format seperti 3 pangkat 4 = 81. Konsep yang dipakai di sini

adalah enkapsulasi data dengan setter, lalu proses logic dilakukan di dalam class, bukan di main. Ini menunjukkan pemisahan tugas antara penyimpanan data (atribut), logika (method hitung), dan output (method tampilkan).

E. KESIMPULAN

Pada modul ini, dipelajari konsep membungkus dengan membuat atribut private dan method getter/setter, serta method overloading. Praktik juga ada pembuatan operasi pada objek, seperti menghitung luas segitiga dan nilai pangkat, dengan method yang mengakses atau memanipulasi atribut objek. praktik menggunakan class, objek, dan method untuk mengatur data dan operasi secara terstruktur.

F. LAMPIRAN